

Draw It or Lose It
CS 230 Project Software Design
Version 1.2

Table of Contents

CS 230 Project Software Design 1

Table of Contents 1

Document Revision History 2

Executive Summary 2

Requirements 3

Design Constraints 3

System Architecture View 4

Domain Model 4

Evaluation 6

Recommendations 10

Document Revision History

Version	Date	Author	Comments
1.0	09/17/2026	Hiram Sanchez	The initial draft includes completed sections for the Executive Summary, Design Constraints, System Architecture View, and Domain Model for One.
1.1	10/2/2026	Hiram Sanchez	Added the completed Evaluation table (server side, client side, and development tools for Mac, Linux, Windows, and mobile) for Project Two.
1.2	10/12/2026	Hiram Sanchez	The Recommendations section for Project Three, covering platform, architecture, storage, memory, distributed systems, and security, has been completed.

Executive Summary

Currently The Gaming Room has their game Draw It or Lose It available for android devices only. This limits our users to only being able to play on one type of device and does not allow them to play against one another. The Gaming Room would like CTS to recreate this application so that it can be accessed through any device that is able to connect to a web browser. The Gaming Room wants us to maintain the functionality of their game consisting of 4 rounds of one minute each where teams compete to see who can guess what is being drawn before time is up.

CTS is accomplishing this by developing a collection of simple OOP Java Classes. Each object can relate back to something in our game we want to represent. There will be many instances of a Game object that contain a minimum of one Team object, and many Team objects can contain a minimum of one Player object. We have created an Entity class that all objects will inherit from that will hold information that we know every object will have such as its id and name. We can then extend this to other properties as needed. We will also have one instance of our GameService class which will be used to create games and provide ids. Our service will help us ensure that no duplicate names exist between games, teams, and players, and that we only have one instance of our game(s) in memory. As we continue to build on this concept, we can expand this idea.

This will be the starting point of our prototype that we will build to convert Draw It or Lose It into a web-based application. Once this application has more features implemented, we will begin deploying it onto multiple servers. The Gaming Room should note that when we say one instance in memory, we mean this application. In the Design Constraints, we will explain what that means when we convert our game to run on many servers.

Requirements

To make our Gaming Room able to become more then an android application we will need to Draw It or Lose It. This needs to be converted to a web-based game that can be accessed from any browser. Allowing it to have multi-team/multi-player games. With prechecked game and team names that are unique and 1 data source for games.

These requirements are met by Java Game, Team, and Player objects. Each child of an Entity object which will provide ids and name properties. Our GameService singleton will hold one instance of our data and give each id their unique number. By using the Iterator design pattern, we can check that no duplicate names exist.

Design Constraints

- **Cross-platform delivery: We want to deliver Draw It or Lose It to any device that may have our player on it.** Phone, tablet, or web browser. This limits our ability to develop our front end to be agnostic to which platform our user is on. This also forces us to create a back-end service that returns common information (i.e., JSON) to all platforms.

- **Timing-sensitive, shared game state:** There will always be a time for each puzzle being drawn. Once that puzzle is finished (30 second timer), the other teams will have a designated amount of

time to guess that puzzle (15 seconds). Each player will need to know when their turn is to guess/draw. This creates a limitation if we were to have this on the web. All our timers would need to happen on the server, not for each client. Otherwise, we would have inconsistencies on who has finished their round.

- **"Single instance in memory" across multiple servers:** Having one instance of something in memory is quite easy. Since we have our service set to be a singleton, we always will only have one instance. But we will need to keep this constraint in mind if we deploy our web app. We will have multiple instances of our web app running on different servers. If we were to use these multiple instances, we would want to make sure that they were not creating the same IDs for games/teams/players. If our service is trying to ensure a user's name is not taken by someone else. We would not want them to be able to take the same name on another server. When deploying multiple servers, we will want to use a central data source to determine ID and Name uniqueness.

- **Concurrent access:** Since our game allows multiple users to play, we want to be aware of multi-threading issues. If someone is trying to create a game, they may be doing it at the same time as another user. Our services will need to account for this if we allow multiple users to play on our web application.

- **Security over the network:** Our data is now going to be sent across the network. We will want to secure our data when traveling between client and server. We don't need to worry about changing our class design that was shown above in the Domain Model. We can discuss more in depth about our limitations when developing on the web in our recommendations.

System Architecture View

There will be 3 layers to Draw It or Lose It. The first is the desktop or mobile display. The display layer contains no information; it simply displays information from the application server. The second layer will be a Java-based application server (i.e., Apache Tomcat) running on Linux. This layer will run our GameService, Game, Team, and Player object from the Domain Model. We will expose these to our clients through a REST call. The final layer will be our database that stores games, teams, and players. By storing our information, we can keep games open after a server restart and allow multiple servers to access our games if we ever need to.

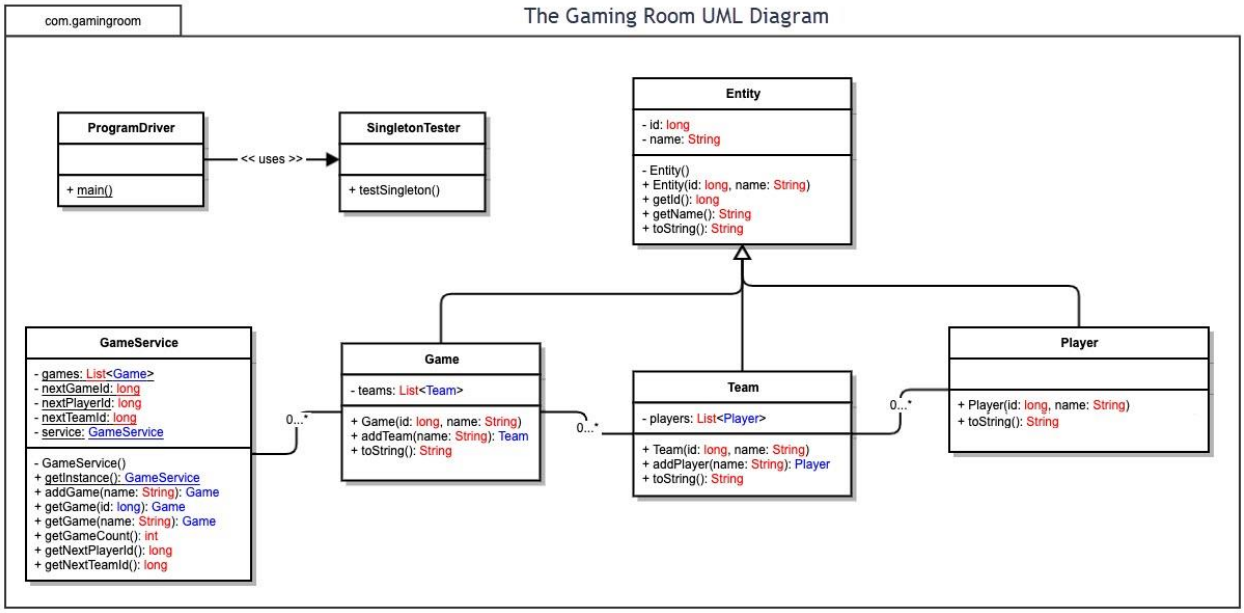
Our system will look something like this: Clients (desktop/mobile) -> [HTTPS/REST call] -> Application Server (GameService, Game, Team, and Players) -> [SQL/JDBC] -> Database (games, teams, and players). As you can see, our system is platform-agnostic because any client can connect to the application server and get the same results. By having only one point of contact between our client and database, we satisfy the design constraint of always having a sole source of truth for all our objects. This will remain true even if we run multiple instances of our server because only our database will hold our information.

Domain Model

They all have an ID (a unique number) and a display name. Since we know they all share these attributes, we created an Entity class that holds these fields along with public getter methods (getId(), getName()) and a toString() method. By using Inheritance, Game, Team, and Player to extend Entity and automatically inherit all shared attributes and behaviors without code duplication. Through Entity, we apply Abstraction by exposing public accessor methods that allow other classes to retrieve an entity's name and ID without revealing how those attributes are stored internally. We also utilize Polymorphism because Game, Team, and Player derive from Entity, external callers and collections can treat them uniformly as Entity instances when executing inherited methods like getId(), getName(), or toString(). By providing only public getters and no setter methods, we enforce Encapsulation, ensuring an entity's name or ID cannot be modified once created. The default Entity() constructor is marked private (-) to prevent empty instantiation, requiring all child objects to invoke the public parameterized constructor super (id, name) during instantiation.

The association lines shown below the classes have a 0..* cardinality because each object holds a list of other objects. For example, a Game holds a List<Team> called teams, meaning one game can contain many teams or none. A Team holds a List<Player> called players, meaning one team can host many players. Our GameService also maintains an association with Game through its internal List<Game> games collection. When adding a team to a game (addTeam(name)) or a player to a team (addPlayer(name)), we use the Iterator Pattern to check if a team or player with that name already exists. This directly fulfills our client's requirement that all game, team, and player names must remain unique.

Our GameService is positioned separately because it follows the Singleton design pattern. We set the GameService() constructor to private so no other instances can be created, storing our single instance in a static variable. Calling getInstance() always returns that same instance in memory. Centralizing sequence counter variables inside this single instance guarantees that every game, team, and player receives a unique ID across the application. At the top of our UML diagram, ProgramDriver connects to SingletonTester via a <<uses>> dependency, representing execution calls made to verify that our singleton maintains memory consistency at runtime.



Evaluation

Development Requirements	Mac	Linux	Windows	Mobile Devices

Server Side	Hosting: Not really a great option. Since MacOS Server has been phased out, you really have no options for hosting your site on a server that will be used by thousands of people. Also need apple hardware which cannot be easily racked or placed in the cloud. Advantages: Good enough to host a copy of your server for a developer. Weaknesses: Hardware costs can't scale extremely high, poor cloud and containerization support. Licensing: Comes with MacOS. So, it's the most expensive option per server when factoring in hardware.	Hosting: Linux is the hosting platform of choice. Java can run on Apache Tomcat or Nginx, allowing us to make REST calls. Linux can run in containers and on all major clouds. Advantages: It's lightweight. Has good stability. Can scale out multiple servers—ability to script Linux deployments. Weaknesses: Admin knowledge takes time to learn. Free versions rely on community support. Licensing: it's Free. Ubuntu, Debian, and Rocky Linux are all free. You can pay for a support line with Red Hat Enterprise Linux and SUSE, but it is not required.	Hosting: Can run the site on a Windows server using IIS. Capable running Java and containers. Microsoft provides good support when deploying to Azure. Advantages: Graphical administration, Active Directory, and vendor support. Weaknesses: More resource intensive. Needs more patching and restarting. Cost to scale. Licensing: Windows Server can get expensive; you will need to look up the current costs. Depending on licensing, they may need CALS as well. Windows Server will be the most expensive server for OS to work with; make sure you get an up-to-date cost from Microsoft before quoting.	Hosting: No, they would not work as servers. With iOS and Android, we must deal with battery and unreliable network connections. Background tasks are limited, so they are not suited as hosts. Advantages: They can act as clients to access our server. Weaknesses: They can't host our application or support a large player count. Licensing: The server OS would be free since no server would run on a phone. But I would still need a server running Linux or Windows.

Client Side	Features: They will be using Safari, Chrome, or Firefox. You would not create a client for this. Compatible with: You should test your responsive HTML in Safari because it runs on the WebKit engine, just like Chrome. Cost/Time: Low cost; you would only need to spend time testing on Mac computers or a website that lets you do this. Knowledge: Normal web dev knowledge but know Safari's quirks.	Features: Fewer users playing the game. They will be using Chrome, Chromium, and Firefox. Compatibility: Write code to Web standards and ensure your game runs on Chrome and Firefox. Cost/Time: Lowest of the desktop platforms due to less testing. Expertise: Same as web development.	Features: This will have the most desktop users. They will be browsing with Edge, Chrome, or Firefox. Compatibility: Test the latest browser first: Edge and Chrome, then Firefox. Cost/ Time: Low cost but will take the most time to test all browsers. Expertise: You can do this with general web development knowledge.	Features: Mobile phones and tablets running IOS safari and Android Chrome. Using a responsive site would allow us to drop our current Android app. Compatibility: Will need to use media queries for responsive sites. Allow users to click items using their finger and make sure it runs smoothly on small devices and connections. Since all iOS browsers use WebKit, we should test it on iOS. Cost/Time: Testing will be the most time-consuming. We can test on physical devices or use a service like a Browser Stack. Progressive Web App can make your web app feel like a native app. Knowledge: Building responsive sites and testing on mobile devices.

Development Tools	Languages and tools: Java (Spring Boot/Tomcat) on the back end and HTML, CSS, and JS on the front end. IDEs we could use: IntelliJ IDEA, Eclipse, VS Code, Eclipse VS Code, and Git. **We would also use the Safari Web Inspector.** All of this can be obtained for free. If we used Xcode, we would have to buy it and need a Mac. But only if we want to build a native iOS app. Team impact: Our web developers could do the same job.	Languages and tools: Java and Web languages. Same IDEs used above. IntelliJ IDEA Community, Eclipse, VS Code. Maven or Gradle and git. Also, what would we host our game on? Licensing: All Free Team impact: Developers need not know anything else to test their local machines. Linux server deployments will be easy.	Languages and tools: Java and web languages. You can use IntelliJ, Eclipse, VS Code, and Visual Studio if you decide to deploy a Windows server. Use Chrome DevTools or Edge DevTools. Licensing: The tools are mostly Free. If they want to use Visual Studio, there is a community version that is free for a small number of developers. Paid options are Professional and Enterprise. Team impact: We won't need any additional team members unless the client wants to develop using .NET or on Windows.	Languages and tools: If we were using a web application, we wouldn't need any programming language specific to a mobile OS. We would need to test our site using Chrome DevTools device emulation and physical devices. If we wanted to create a native app for if, iOS, we would need to program in Swift in Xcode. For Android, we could program in Kotlin or Java in Android Studio, both for free. Licensing: Using our website on mobile browsers will be free. If we wanted to publish a native app, we would need to buy a license from Apple ($99 a year and Google Play $25 once. Team impact: We can add web developers to support mobile browsers. If we wanted to build a native app, we would have to hire each platform, which is another reason to stick with a web application.

Recommendations

1. **Operating Platform:** We should host our game on Linux-based servers. There are many Linux distributions, but we believe our server(s) should run Ubuntu Server LTS or Rocky Linux. In our evaluation, we went over the pros of Linux. It's free and works for all cloud providers. It lets us run containers and Java programs. Since our game will let people on Windows, Mac, Android, and iOS all play through our website. We only need to keep up with one type of server. The Gaming Room won't have to maintain a server for each platform. As our game grows, we can add more servers to our pool without paying per server.

2. **Operating Systems Architectures**: Linux has a monolithic kernel. It allows loadable modules to be inserted into the kernel. The kernel handles services for our computer, such as process scheduling and memory management. File systems and networking are all run in kernel space. Our program runs in user space and will request these services through system calls. If our game crashes, it will not crash our operating system. We plan to use Apache Tomcat to host our application on our Linux machine. We will serve our GameService, Game, Team, and Player object over HTTPS using REST calls to our clients. We can run many games at once because Linux has a scheduler that distributes our games across multiple CPU cores. As stated in our concurrency constraint, our GameService needs to be thread safe. We would need to make methods such as addGame, addTeam, and addPlayer synchronized so two users can't create games with the same IDs or names at the same time. Linux supports containers, which lets us run Tomcat and the game in one container and send copies to other servers.

3. **Storage Management**: As you can see, we currently hold all of our Games, Teams, and Players on a List in our GameService. While this is fine for the prototype, we lose all our data when we restart the application. Also, each server will have its own copy of these games/teams/players. We suggest using a relational database. Postgres or MySQL. Using SQL/JDBC through Tomcat. Using a database will let us keep our games persistent across restarts. All servers will see the same games, and we can ensure unique IDs and Names in one location. We can do this by creating unique constraints in the database and using an ID sequence that automatically generates IDs, rather than storing a counter in memory. Rather than storing a counter in memory. Make sure the Database is on a file system with journaling capabilities. This will help with crash recovery and power loss, as we will be storing many images for our game. Around 200 at 8MB each; that's roughly 1.6GB. You may want to host these images on the file system or use a Content Delivery Network. It's best practice to back up your database often and make sure those backups work. You'll also want to plan growth in the image library and your game history.

4. **Memory Management:** Linux uses virtual memory, meaning it lets the user use more than is installed on the computer. Virtual memory allocates address space for each program. With demand paging, Linux pulls pages into memory only when it needs them. When reading files from the hard drive, Linux will store them in memory if the page cache has enough free space. So commonly used puzzle pictures will be in memory. Our program will use the Java Virtual Machine's built-in garbage collection to collect unused objects in memory. Once a game ends, we can remove all game teams and players that are not currently being used. We want to leave memory available for the operating system's cache. We can do this by setting the Java VM's

maximum heap space. Remove ended games from memory, and don't load all 200 pictures at once. Load the picture you are currently on and have the next one ready. Let multiple games use a picture already stored in memory. Having “1” in memory means it's on one server. If we have multiple servers, our database will be a single in-memory instance. Linux uses swap space as a last resort; we want to try to keep our program out of swap as much as possible since each round is 1 minute.

5. **Distributed Systems and Networks**: Our Draw It or Lose It design involves many clients talking to our server over the internet. These clients can be on any platform and will communicate with our Linux servers through HTTPS using REST. Our server will tell our database using SQL/JDBC. Since all platforms can communicate over TCP/IP and use the same API, one player could be on an iPhone while another is on a Windows computer. As previously mentioned, when designing our constraints, the timer would need to be stored on the server side. This way, no one could have a slower connection or hack the client side to gain a time advantage over other players. By implementing WebSocket's, we can notify players of game changes, such as new guesses or the end of the round.

   We know these dependencies can fail, so here are some ideas for fault tolerance. If a player disconnects from the game, we want our server to wait a brief period to allow them to reconnect and continue playing. We would also want more than 2 servers available to handle user requests. We would do this by using multiple application servers and placing a load balancer in front of them with health checks. If one of our application servers goes down, the others can still handle requests. Now we have a single point of failure: our database. So, we would want to replicate our database with failover or use a database service. Having servers in multiple data centers can also help if something happens to one data center. If our database were down, we would want to show an error to users and prompt them to reconnect.

6. **Security:** Since all your data will travel over the internet, we should use HTTPS (TLS). This means all communication between the client and servers, and between the servers and the database, should be encrypted. On our servers, we can validate requests and use prepared statements to defend SQL Injection. Then we can make sure who is requesting these functions by implementing sessions. Never store passwords in plain text. Use algorithms that hash passwords with a salt; examples include bcrypt and Argon2. Linux also has features to help keep your server safe. You can set up users and groups. Tomcat should run as its own user, not as root, and have SELinux/AppArmor enabled to restrict it from doing anything unnecessary. Security system rules should only open the ports you need, such as 443. Use SSH keys for admins and make sure to apply patches. If many children play this game, we want to minimize the PII we collect and limit who can access it. Logging will also help us detect any malicious activity.